EX.NO: 3 PROGRAMS ON CALLBACK, PROMISE AND ASYNC/AWAIT

DATE:

AIM:

To implement asynchronous operations using Callbacks, Promises, and Async/Await for handling tasks such as order processing, file uploading, processing, and report generation in JavaScript.

1. E-Commerce Order Processing:

An e-commerce platform has addToCart(product, callback / promise / async) – adds a product to the cart (1 sec delay), placeOrder(order, callback / promise / async) – places the order (2 sec delay), sendConfirmationEmail(order) – sends order confirmation email.

PROGRAM:

----------***Using Callback*** ----------

```
console.log("Esakki Durai M - 24104105");
function addToCart(product, callback) {
  console.log("Adding product to cart...");

  setTimeout(() => {
    console.log(product + " added to cart");
    callback(product);
  }, 1000);
}
function placeOrder(product, callback) {
  console.log("Placing order...");

  setTimeout(() => {
    const order = {
      orderId: 101,
      product: product
    };
    console.log("Order placed successfully");
    callback(order);
  }, 2000);
}
function sendConfirmationEmail(order) {
  console.log(
    "Confirmation email sent for order ID:",
```

```
        order.orderId );
}
addToCart("Laptop", function(product) {
    placeOrder(product, function(order) {
        sendConfirmationEmail(order);
    });
});
```

Using Promises

```
console.log("Esakki Durai M - 24104105");
function addToCart(product) {
    return new Promise((resolve, reject) => {
        console.log("Adding product to cart...");

        setTimeout(() => {
            console.log(product + " added to cart");
            resolve(product);
        }, 1000);
    });
}
function placeOrder(product) {
    return new Promise((resolve, reject) => {
        console.log("Placing order...");
        setTimeout(() => {
            const order = {
                orderId: 101,
                product: product
            };
            console.log("Order placed successfully");
            resolve(order);
        }, 2000);
    });
}
function sendConfirmationEmail(order) {
    console.log(
        "Confirmation email sent for order ID:",
        order.orderId
    );
}
addToCart("Laptop")
    .then(product => placeOrder(product))
    .then(order => sendConfirmationEmail(order))
    .catch(error => console.log(error));
```

----------***Using Async/Await*** ----------

```
console.log("Esakki Durai M - 24104105");
function addToCart(product) {
   return new Promise((resolve, reject) => {
      console.log("Adding product to cart...");

      setTimeout(() => {
         console.log(product + " added to cart");
         resolve(product);
      }, 1000);
   });
}
function placeOrder(product) {
   return new Promise((resolve, reject) => {
      console.log("Placing order...");
      setTimeout(() => {
         const order = {
            orderId: 101,
            product: product
         };
         console.log("Order placed successfully");
         resolve(order);
      }, 2000);
   });
}
function sendConfirmationEmail(order) {
   console.log(
      "Confirmation email sent for order ID:",
      order.orderId
   );
}
async function processOrder() {
   try {
      const product = await addToCart("Laptop");
      const order = await placeOrder(product);
      sendConfirmationEmail(order);
   } catch (error) {
      console.log(error);
   }
}
processOrder();
```

OUTPUT:

```
PS C:\Esakki_durai> node new.js
Esakki Durai M - 24104105
Adding product to cart...
Laptop added to cart
Placing order...
Order placed successfully
Confirmation email sent for order ID: 24104105
```

2. File Upload and Processing:

In a document management system uploadFile(file, callback / promise / async) – uploads a file (2 sec delay), processFile(file, callback / promise / async) – processes the uploaded file (3 sec delay), generateReport(file) – generates a report.

PROGRAM:

----------***Using Callback*** ----------

```
console.log("Esakki Durai M - 24104105");
function uploadFile(file, callback) {
    console.log("Uploading file...");
    setTimeout(() => {
        console.log(file + " uploaded successfully");
        callback(file);
    }, 2000);
}
function processFile(file, callback) {
    console.log("Processing file...");

    setTimeout(() => {
        console.log(file + " processed successfully");
        callback(file);
    }, 3000);
}
function generateReport(file) {
    console.log("Report generated for:", file);
}
uploadFile("document.pdf", function(file) {
    processFile(file, function(processedFile) {
        generateReport(processedFile);
    });
});
```

----------***Using Promises*** ----------

```
console.log("Esakki Durai M - 24104105");
function uploadFile(file) {
    return new Promise((resolve, reject) => {
        console.log("Uploading file...");
        setTimeout(() => {
            console.log(file + " uploaded successfully");
            resolve(file);
        }, 2000);
    });
}
function processFile(file) {
    return new Promise((resolve, reject) => {
        console.log("Processing file...");

        setTimeout(() => {
            console.log(file + " processed successfully");
            resolve(file);
        }, 3000);
    });
}
function generateReport(file) {
    console.log("Report generated for:", file);
}
uploadFile("document.pdf")
    .then(file => processFile(file))
    .then(processedFile => generateReport(processedFile))
    .catch(error => console.log(error));
```

----------***Using Async/Await*** ----------

```
console.log("Esakki Durai M - 24104105");
function uploadFile(file) {
    return new Promise((resolve, reject) => {
        console.log("Uploading file...");
        setTimeout(() => {
            console.log(file + " uploaded successfully");
            resolve(file);
        }, 2000);
    });
}
function processFile(file) {
    return new Promise((resolve, reject) => {
        console.log("Processing file...");
```

```
        setTimeout(() => {
            console.log(file + " processed successfully");
            resolve(file);
        }, 3000);
    });
}
function generateReport(file) {
    console.log("Report generated for:", file);
}
async function handleFile() {
    try {
        const uploadedFile = await uploadFile("document.pdf");
        const processedFile = await processFile(uploadedFile);
        generateReport(processedFile);
    } catch (error) {
        console.log(error);
    }
}
handleFile();
```

OUTPUT:

```
PS C:\Esakki_durai> node new.js
Esakki Durai M - 24104105
Uploading file...
document.pdf uploaded successfully
Processing file...
document.pdf processed successfully
Report generated for: document.pdf
```

Problem Undersatnding & Approach (10)	Implementation & Execution (15)	Tool/Technology Usage (10)	Code Quality & Documentation (5)	Viva (5)	Time Management (5)	Total (50)

RESULT:

The asynchronous operations were successfully implemented using Callbacks, Promises, and Async/Await, ensuring that each task executed in the correct order.